OS & SP

Name : D. Teja Sadwika **Roll No :** 2520030536

SKILL – 17

1.Skill17.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 100
#define MAX_CMD 256

typedef struct {
    int job_id;
    pid_t pid;
    char command[MAX_CMD];
    int running;
```

```
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;

/* Add background job */
void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS) {
        printf("Job table full!\n");
        return;
    }

    jobs[job_count].job_id = next_job_id++;
    jobs[job_count].pid = pid;

    strncpy(jobs[job_count].command, command, MAX_CMD -
1);
```

```
    jobs[job_count].command[MAX_CMD - 1] = '\0';
    jobs[job_count].running = 1;

    printf("[%d] %d Running %s\n",
           jobs[job_count].job_id,
           (int)jobs[job_count].pid,
           jobs[job_count].command);

    job_count++;
}

/* Update job status */
void update_jobs(void)
{
    for (int i = 0; i < job_count; i++) {
        if (!jobs[i].running)
            continue;
```

```
        int status;

        pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

        if (result == jobs[i].pid) {
            jobs[i].running = 0;

            printf("[%d] Done %s\n",
                   jobs[i].job_id,
                   jobs[i].command);
        }
    }
}

/* List active jobs */
void show_jobs(void)
{
    update_jobs();
```

```
    printf("\n----- Active Jobs -----\n");

    for (int i = 0; i < job_count; i++) {
        if (jobs[i].running) {
            printf("[%d] PID=%d Running %s\n",
                   jobs[i].job_id,
                   (int)jobs[i].pid,
                   jobs[i].command);
        }
    }

    printf("----------------------\n");
}

/* Bring job to foreground */
void foreground_job(int job_id)
{
```

```
for (int i = 0; i < job_count; i++) {
    if (jobs[i].job_id == job_id && jobs[i].running) {

        printf("Bringing job [%d] to foreground...\n", job_id);
        printf("Command: %s\n", jobs[i].command);

        /* Wait for selected job */
        int status;

        if (waitpid(jobs[i].pid, &status, 0) == -1) {
            perror("waitpid");
            return;
        }

        jobs[i].running = 0;

        printf("[%d] Completed\n", job_id);
        return;
```

```
        }
    }

    printf("Job [%d] not found or already completed.\n", job_id);
}

int main(void)
{
    char input[MAX_CMD];

    printf("OSSP Program 17 - Job Control\n");
    printf("Commands:\n");
    printf(" jobs       -> list active jobs\n");
    printf(" fg <id>    -> bring job to foreground\n");
    printf(" exit       -> exit shell\n\n");

    while (1) {
        update_jobs();
```

```
printf("shell> ");
fflush(stdout);

if (fgets(input, sizeof(input), stdin) == NULL)
    break;

input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0)
    continue;

/* Exit */
if (strcmp(input, "exit") == 0)
    break;

/* Jobs command */
if (strcmp(input, "jobs") == 0) {
```

```
    show_jobs();
    continue;
}

/* Foreground command */
if (strncmp(input, "fg ", 3) == 0) {
    int job_id = atoi(input + 3);
    foreground_job(job_id);
    continue;
}

/* Detect background & */
int background = 0;
int len = (int)strlen(input);

if (len > 0 && input[len - 1] == '&') {
    background = 1;
    input[--len] = '\0';
```

```
    while (len > 0 &&
           (input[len - 1] == ' ' ||
            input[len - 1] == '\t')) {
        input[--len] = '\0';
    }
}

if (input[0] == '\0')
    continue;

/* Save command before creating child */
char command_copy[MAX_CMD];

strncpy(command_copy, input, MAX_CMD - 1);
command_copy[MAX_CMD - 1] = '\0';

pid_t pid = fork();
```

```
if (pid < 0) {
    perror("fork");
    continue;
}

/* Child process */
if (pid == 0) {
    char *args[64];
    int argc = 0;

    char *token = strtok(input, " \t");

    while (token != NULL && argc < 63) {
        args[argc++] = token;
        token = strtok(NULL, " \t");
    }
```

```
    args[argc] = NULL;

    if (argc == 0)
        _exit(0);

    execvp(args[0], args);

    perror("execvp");
    _exit(1);
}

/* Parent process */
else {
    if (background) {
        add_job(pid, command_copy);
    } else {
        int status;
```

```
            if (waitpid(pid, &status, 0) == -1)
                perror("waitpid");
        }
    }
}

printf("\nExiting shell...\n");

/* Wait for remaining background jobs */
for (int i = 0; i < job_count; i++) {
    if (jobs[i].running) {
        waitpid(jobs[i].pid, NULL, 0);
        jobs[i].running = 0;
    }
}

return 0;
}
```

OUTPUT :

```
sadwika@DESKTOP-8CQAO1L:~$ nano skill17.c
sadwika@DESKTOP-8CQAO1L:~$ gcc -Wall -Wextra -g skill17.c -o skill17
sadwika@DESKTOP-8CQAO1L:~$ ./skill17
OSSP Program 17 - Job Control
Commands:
 jobs       -> list active jobs
 fg <id>    -> bring job to foreground
 exit       -> exit shell

shell> jobs

----- Active Jobs -----
-----------------------
shell> fg 1
Job [1] not found or already completed.
shell> exit

Exiting shell...
sadwika@DESKTOP-8CQAO1L:~$
```